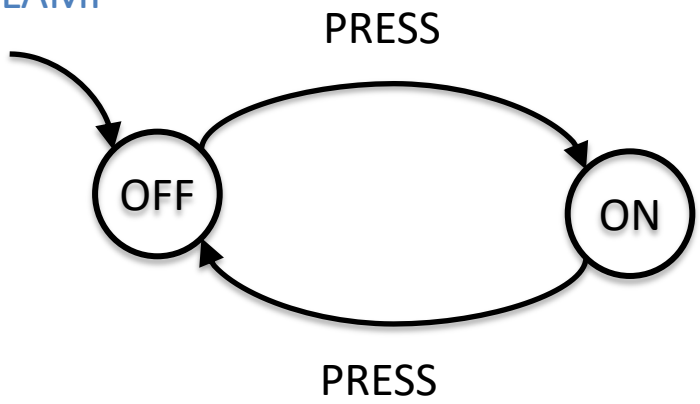
2.

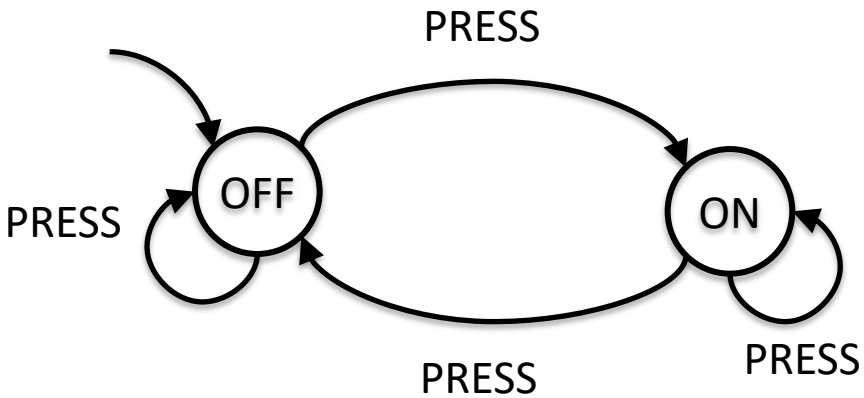
NON-DETERMINISTIC FINITE AUTOMATA (NFA)

Example: A lamp

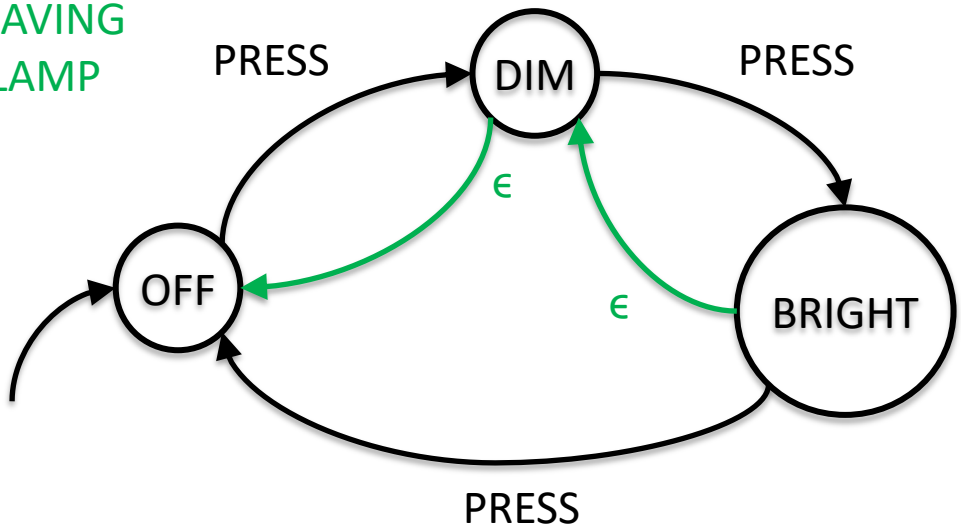
BASIC LAMP



FAULTY LAMP



ENERGY SAVING SMART LAMP

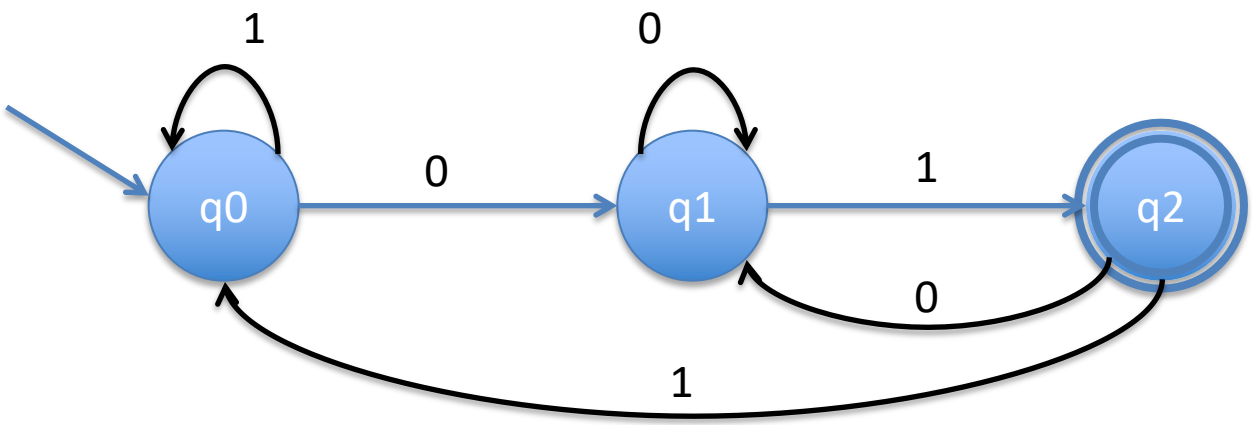


ϵ are ”internal actions”

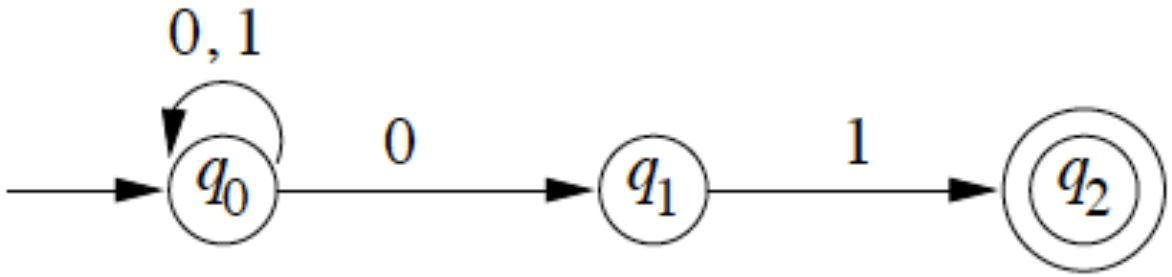
Why non-determinism?

- Language of all strings over Σ={0,1} ending in 01

DFA

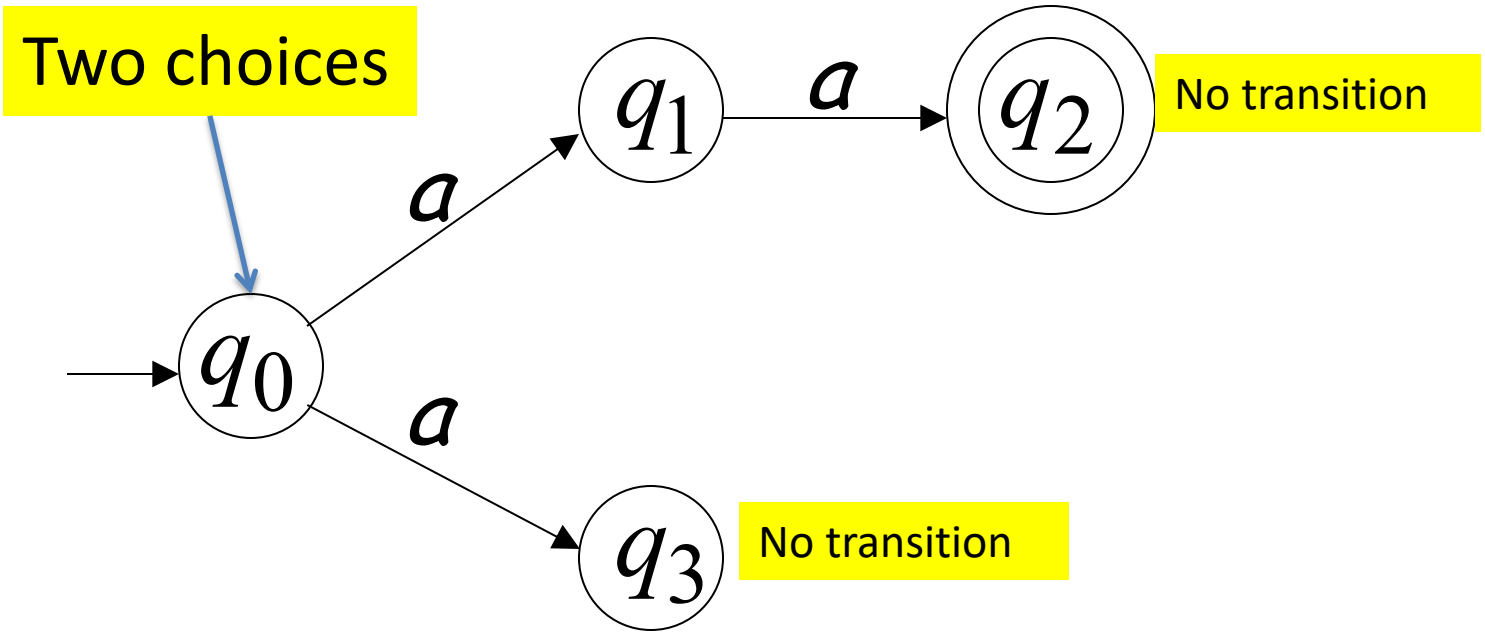


NFA



Non-deterministic Finite Automata (NFA)

Alphabet = {*a*}

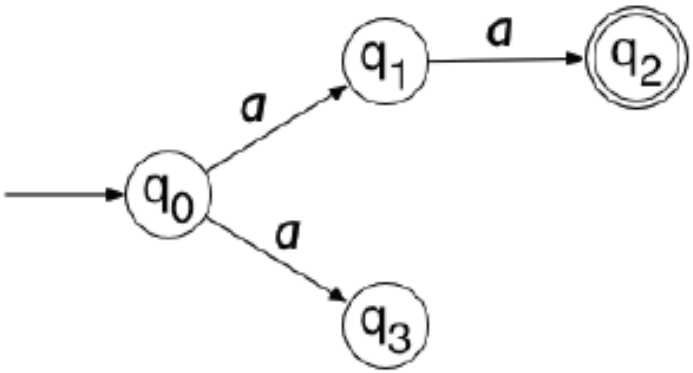


Non-deterministic Finite Automata (NFA)

- An NFA *accepts* a string *w* if and only if there is at least one computation of the NFA that accepts *w* i.e. a computation that
  - consumes the entire input, and
  - stops in a final state

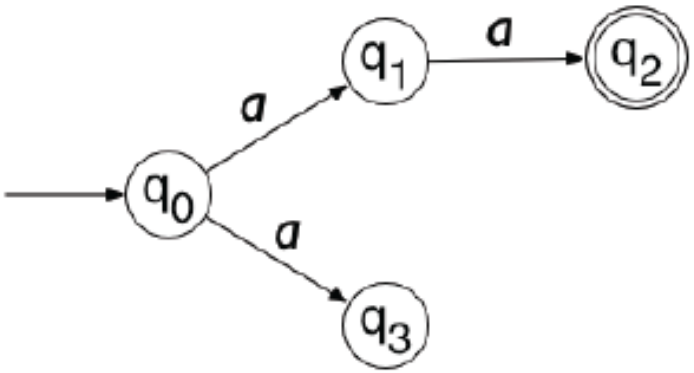
Examples

Input	State sequence	Input consumed?	Stops in final state?	Accepted?	Accepted by NFA?
aa	$q_0q_1q_2$				
aa	q_0q_3				
a	q_0q_1				
a	q_0q_3				
aaa	$q_0q_1q_2$				
aaa	q_0q_3				



Examples

Input	State sequence	Input consumed?	Stops in final state?	Accepted?	Accepted by NFA?
aa	$q_0q_1q_2$	✓	✓	✓	✓
aa	q_0q_3	✗	✗	✗	
a	q_0q_1	✓	✗	✗	✗
a	q_0q_3	✓	✗	✗	
aaa	$q_0q_1q_2$	✗	✓	✗	✗
aaa	q_0q_3	✗	✗	✗	



Formal definition

A non-deterministic finite automaton consists of

- Q: a finite set of states
- Σ: a finite set of input symbols, an alphabet
- $\delta: Q \times \Sigma \rightarrow P(Q)$: a transition function
  for each state q in Q and each symbol a in Σ it determines a set of new states δ(q,a)
- q_0: the initial state; an element of Q
- F: the final states; a subset of Q

Often written as $A = (Q, \Sigma, \delta, q_0, F)$

The language of the NFA

By induction on the length of the string w:

- $\delta^*(q,\varepsilon) = \{q\}$
- $\delta^*(q,aw) = \delta^*(q_1,w) \cup \ldots \cup \delta^*(q_k,w)$

  where $\delta(q,a) = \{q_1,\ldots,q_k\}$

The language of $A = (Q, \Sigma, \delta, q_0, F)$

$L(A) = \{w \mid \delta^*(q_0,w)$ contains a state from $F\}$

so at least one of the attempts must lead to a final state

Try Out

- Consider the alphabet {0,1,2}. Specify an NFA for the set of strings such that the final digit has appeared before – try to take advantage of non-determinism as much as possible.

FROM DFA TO NFA

The embedding construction

- Let $A = (Q, \Sigma, \delta, q_0, F)$ be a DFA.
- Construct the NFA $A = (Q, \Sigma, \delta, q_0, F)$ as follows:
  - Q equals Q
  - q_0 is the set q_0
  - F equals F
  - $\delta: Q \times \Sigma \rightarrow P(Q)$ is constructed from $\delta: Q \times \Sigma \rightarrow Q$

$$\boxed{\text{if } \delta(q,a) = p \text{ then } \delta(q,a) = \{p\}}$$

Theorem

- Let $A = (Q, \Sigma, \delta, q_0, F)$ is a DFA and assume that the NFA $A = (Q, \Sigma, \delta, q_0, F)$ is constructed by the embedding construction.
- Then $L(A) = L(A)$

- Proof: It is sufficient to show that $\delta^*(q,w) = \{\delta^*(q,w)\}$ for all q and w

Proof

- To prove $\delta^*(q,w) = \{\delta^*(q,w)\}$ we proceed by induction on the length of the string w

$\delta^*(q,\varepsilon) = \{q\}$

$\delta^*(q,wa) = \delta(q_1,a)$

where $\delta^*(q,w) = \{q1\}$

NFA

$\{\delta^*(q,\varepsilon\}\} = \{q\}$

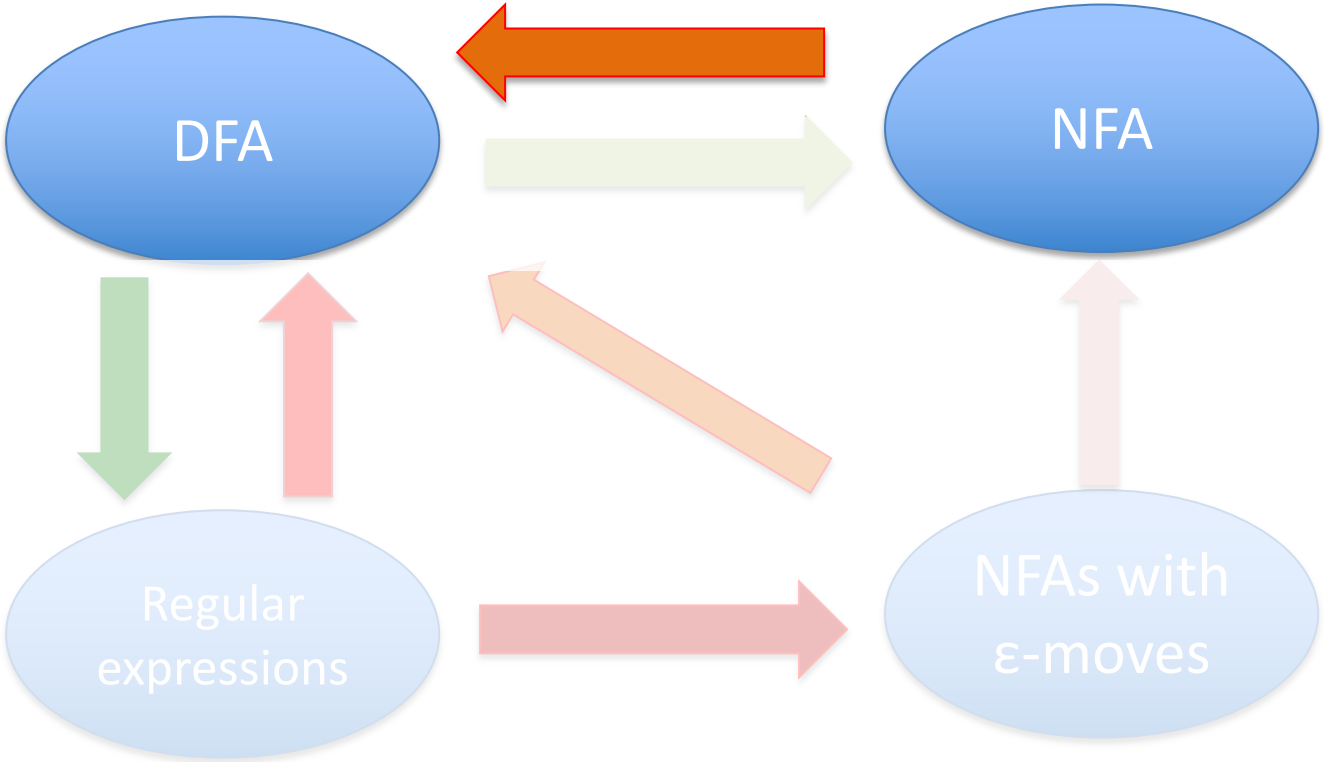
$\{\delta^*(q,wa)\} = \{\delta(q1,a)\}$

where $q1=\delta^*(q,w)$

(by induction on w)

DFA

The big picture



The subset construction

- For each NFA we construct a DFA that accepts the same language
  - The DFA may have 2^n states when the NFA have n states – exponential blow up!
  - But the good news is that in most cases it will have the same number of states but a larger number of transitions

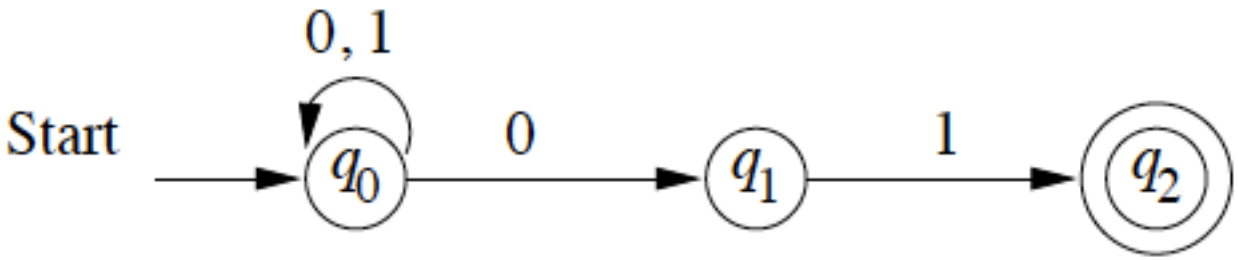
The subset construction

Complexity: $O(2^n)$

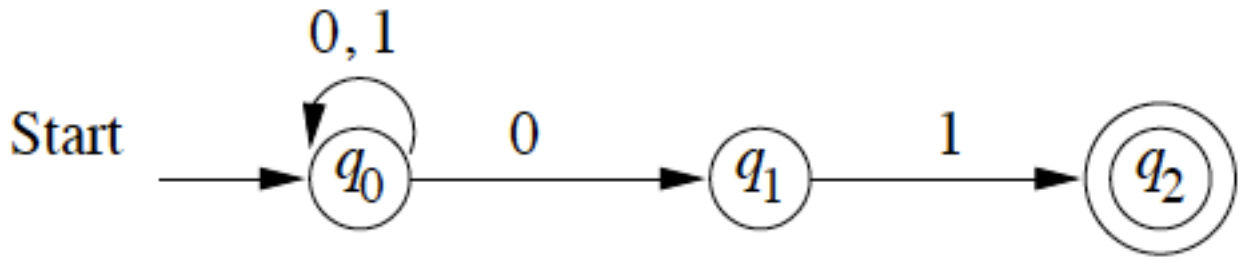
- Let $A_N = (Q_N, \Sigma, \delta_N, q_{N0}, F_N)$ be a NFA.
- Construct the DFA $A_D = (Q_D, \Sigma, \delta_D, q_{D0}, F_D)$ as follows:
  - Q_D is the set of all subsets of Q_N (so $Q_D = P(Q_N)$)
  - q_{D0} is the set $\{q_{N0}\}$
  - F_D is the subsets of Q_N that contains states from F_N
  - $\delta_D: Q_D \times \Sigma \rightarrow Q_D$ constructed from $\delta_N: Q_N \times \Sigma \rightarrow P(Q_N)$

$$\boxed{\delta_D(\{q_1,\ldots,q_k\},a) = \delta_N(q_1,a) \cup \ldots \cup \delta_N(q_k,a)}$$

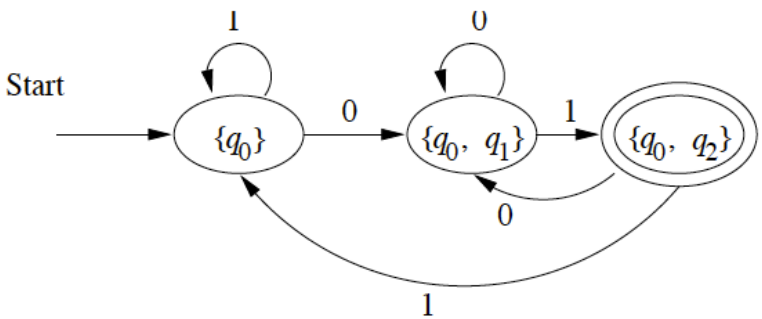
Example



Example



Becomes



	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

all the subsets of $\{q_0,q_1,q_2\}$

Final states

Try Out

- Use the algorithm to convert the following NFA to a DFA

	0	1
→p	{p,q}	{p}
q	{r,s}	{t}
r	{p,r}	{t}
*s	Ø	Ø
*t	Ø	Ø

Is our construction correct?

- Let $A = (Q, \Sigma, \delta_N, q_0, F)$ be an NFA and assume that the DFA $A = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is constructed by the subset construction.
- Then $L(A) = L(A)$

- Proof: It is sufficient to show that

$$\delta^*_N(q,w) = \delta^*_D(\{q\},w)$$

for all q and w

Proof

- To prove $\delta^*_N(q,w) = \delta^*_D(\{q\},w)$ we proceed by induction on the length of the string w

$\delta^*_N(q,\varepsilon) = \{q\}$

$\delta^*_N(q,wa) = \delta_N(q_1,a) \cup \ldots \cup \delta_N(q_k,a)$

where $\delta_N(q,w) = \{q_1,\ldots,q_k\}$

NFA

$\delta^*_D(\{q\},\varepsilon) = \{q\}$

$\delta^*_D(\{q\},wa) = \delta_D(\{q_1,\ldots,q_k\},a)$

where $\delta^*_D(\{q\},w)$

$= \{q_1,\ldots,q_k\}$

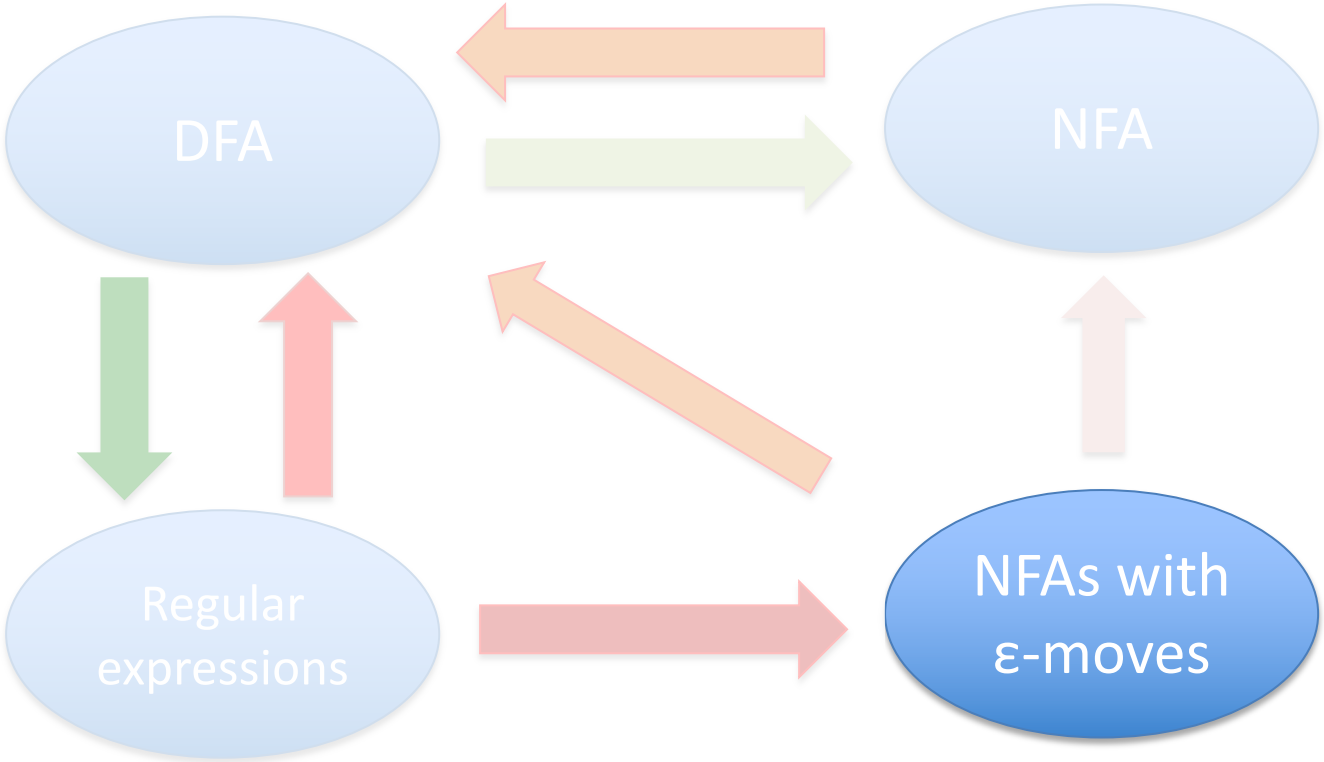
(by induction on w)

DFA

$$\boxed{\delta_N(q_1,a) \cup \ldots \cup \delta_N(q_k,a) = \delta_D(\{q_1,\ldots,q_k\},a)}$$

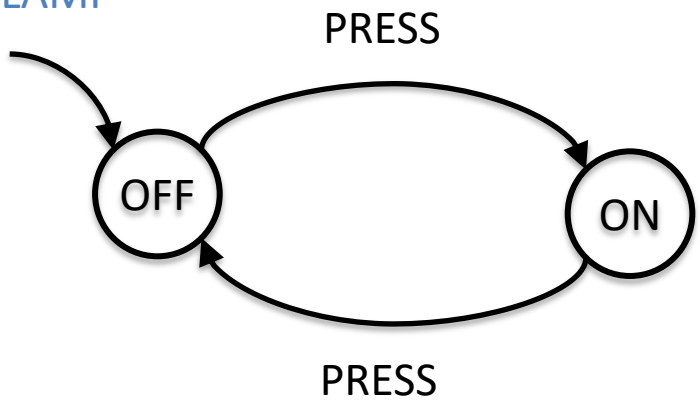
(by definition of δ_D)

The big picture

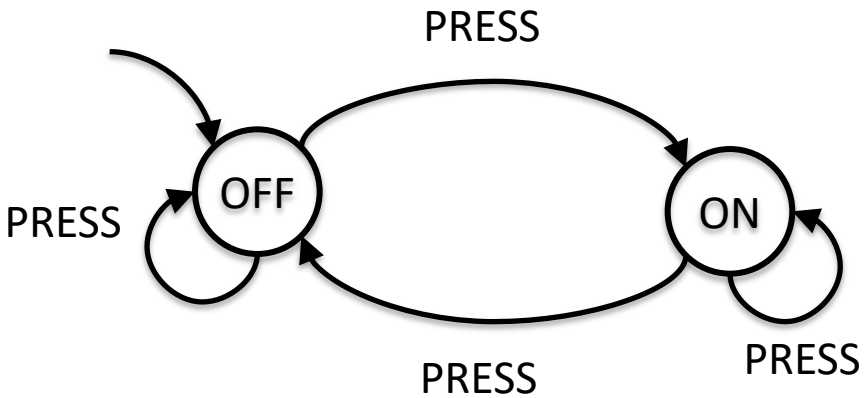


Example: A lamp

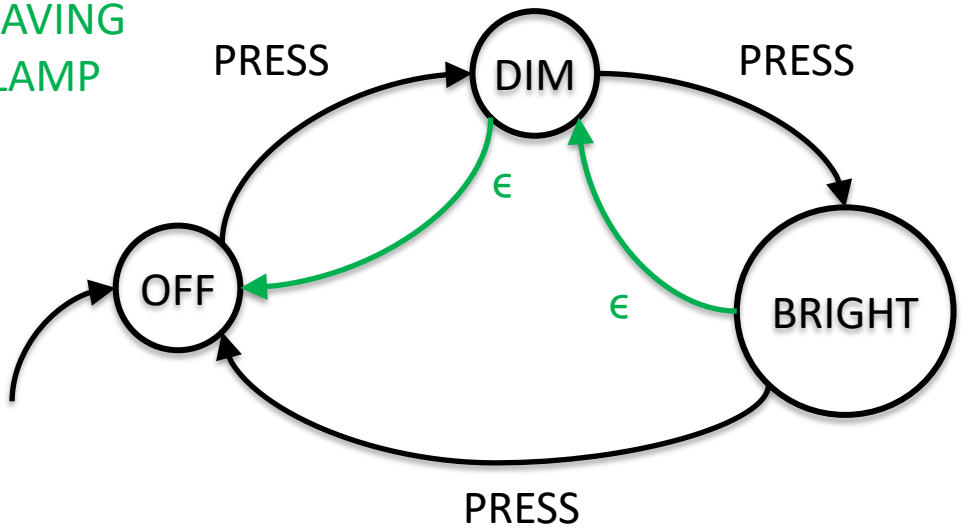
BASIC LAMP



FAULTY LAMP



ENERGY SAVING SMART LAMP



ϵ are "internal actions"

Formal definition

A non-deterministic finite automaton with ε-moves consists of

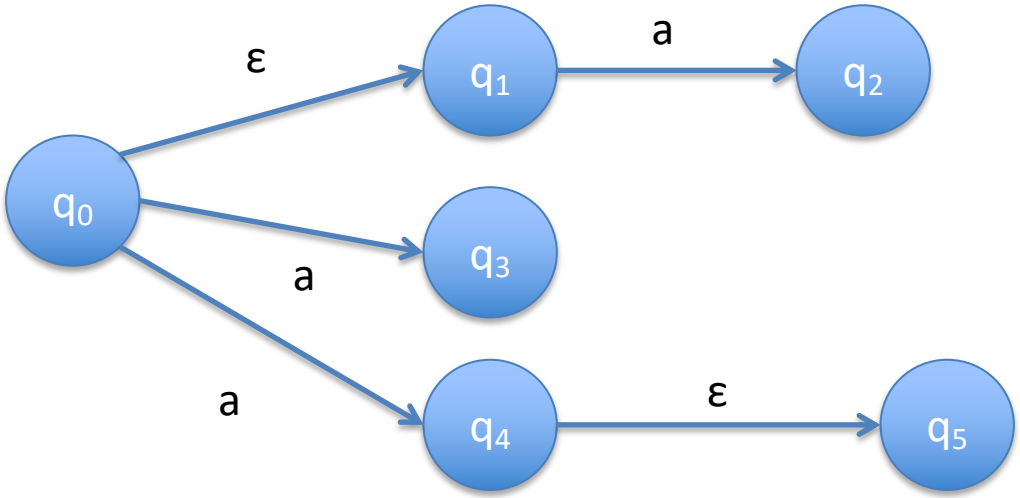
- Q: a finite set of states
- Σ: a finite set of input symbols, an alphabet
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$: a transition function
  for each state q in Q and perhaps a symbol a in Σ it determines a set of new states δ(q,a)
- q_0: the initial state; an element of Q
- F: the final states; a subset of Q

The automaton does not need to read a symbol

Often written as $A = (Q, \Sigma, \delta, q_0, F)$

The language accepted by an ε-NFA

- How can we define the extended transition function δ*? I.e. $\delta^*((q_0,a) = ?$

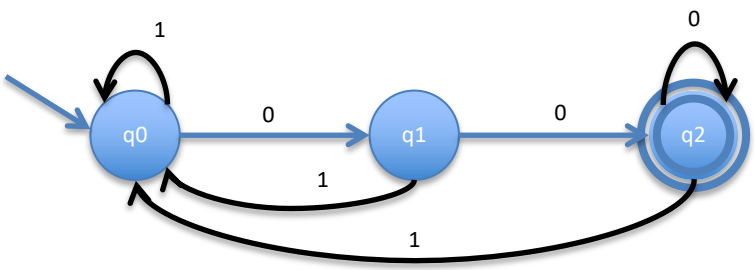


IDEA:

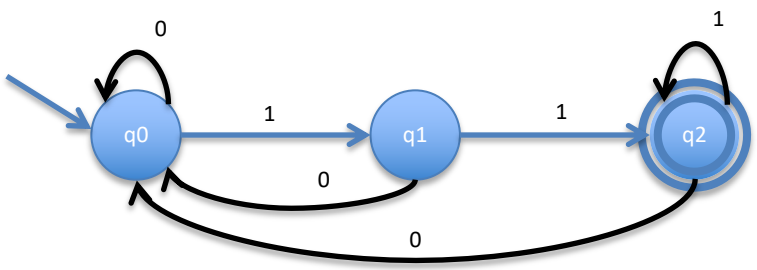
1. follow as many ε-transitions as you want
2. Read a
3. follow as many ε-transitions as you want

Motivation

Alice likes strings ending with 00



Bob likes strings ending with 11



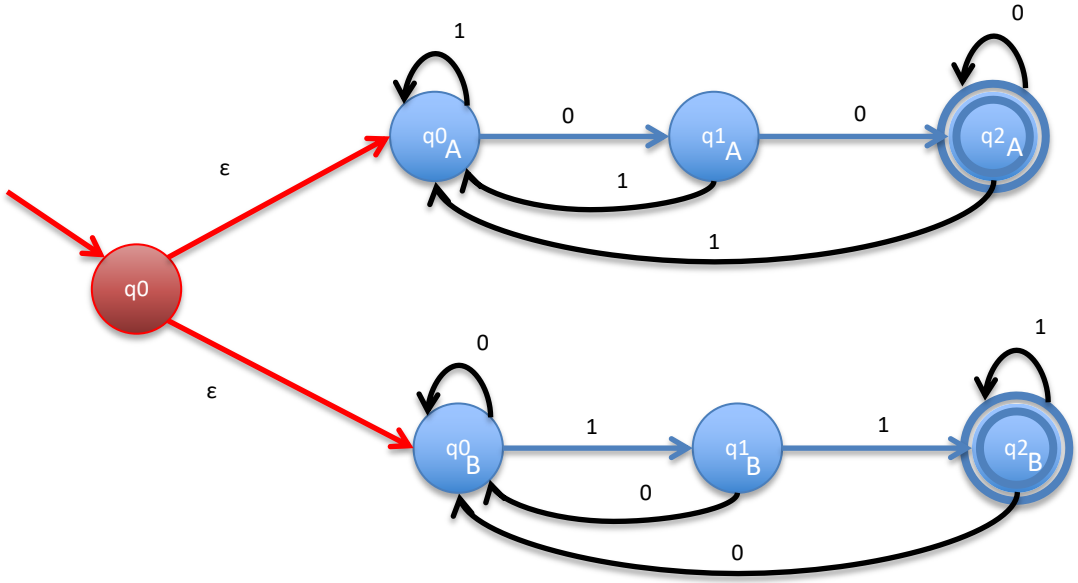
Charlie likes both sets of strings and their DFA representation. He needs to design a DFA for the union of the sets of strings. Bob only knows DFAs.

Same for Danielle… but she knows e-NFA.

Who will have an easier life (i.e. designing the desired automaton)? Try…

Danielle's solution

Danielle can just combine Alice's and Bob's automata like this (changes highlighted in red).

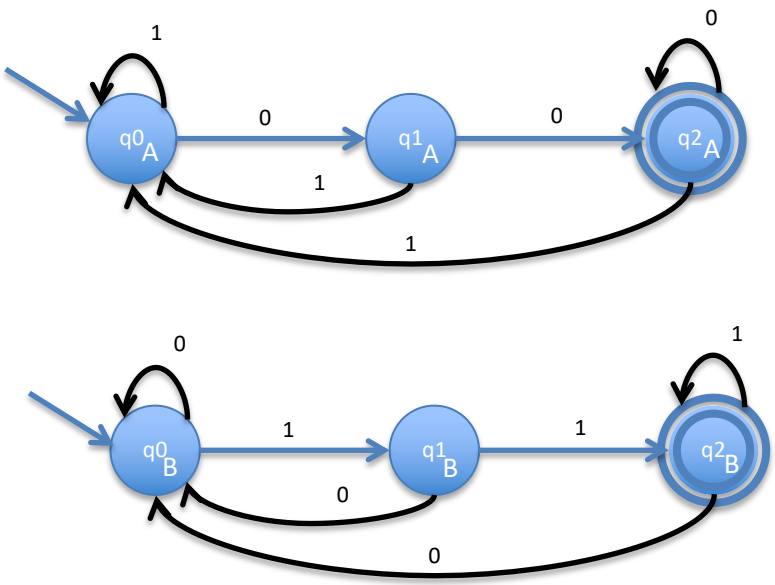


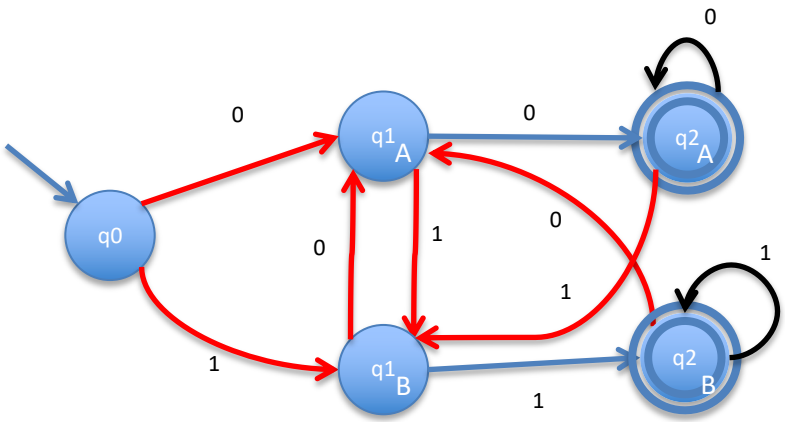
As we shall see in RL3. The union of two languages is easy to design with ε-NFA!

… and if you need a DFA, you can just convert the ε-NFA into a DFA as we shall see.

Bob's solution

Bob cannot easily combine the DFA and will have to design the DFA from scratch or rearrange transitions heavily. See a possible solution with highlighted changes.



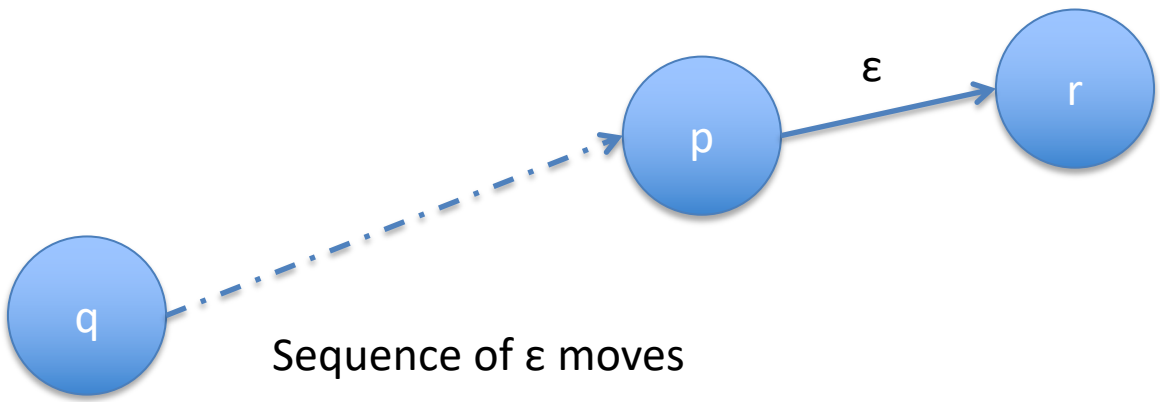


The combination is not really "compositional" as it requires to merge some states and re-arrange transitions. Instead, ε-NFA are more amenable for this kind of composition (language union)

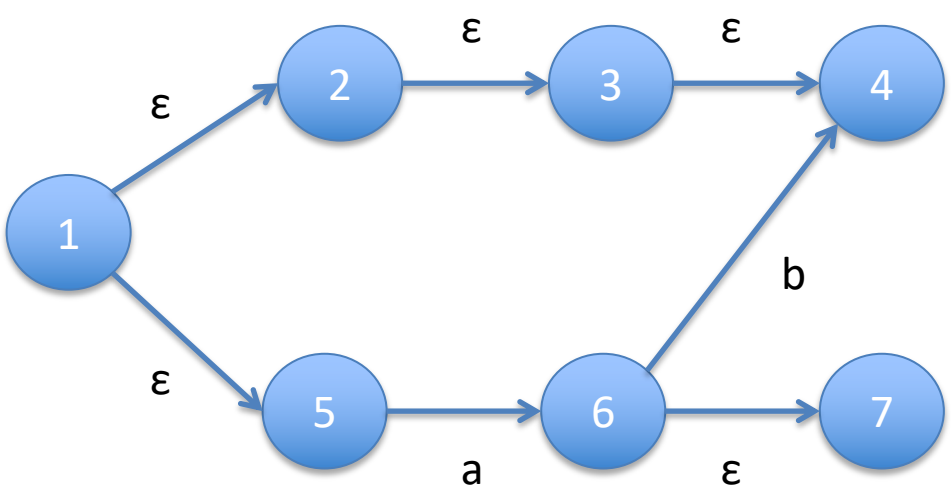
ECLOSE(q) - ε-closure

- q is in ECLOSE(q)
- If p is in ECLOSE(q) and r is in δ(p,ε)

  then r is in ECLOSE(q)

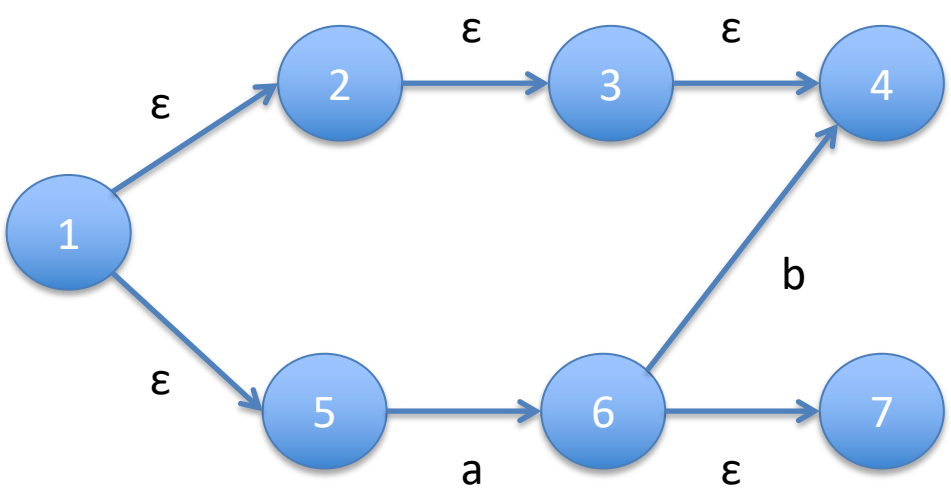


Example



q	ECLOSE(q)
1	
2	
3	
4	
5	
6	
7	

Example



q	ECLOSE(q)
1	{1,2,3,4,5}
2	{2,3,4}
3	{3,4}
4	{4}
5	{5}
6	{6,7}
7	{7}

The language of the ε-NFA

By induction on the length of the string w:

- $\delta^*(q,\varepsilon) = \text{ECLOSE}(q)$
- $\delta^*(q,wa) = \text{ECLOSE}(\delta(q_1,a) \cup \ldots \cup \delta(q_k,a))$

  where $\delta^*(q,w) = \{q_1,\ldots,q_k\}$

Note: ECLOSE is lifted to sets:

$$\text{ECLOSE}(\{q_1',\ldots,q_k'\}) = \text{ECLOSE}(q_1') \cup \ldots \cup \text{ECLOSE}(q_k')$$

The language of $A = (Q, \Sigma, \delta, q_0, F)$

$L(A) = \{w \mid \delta^*(q_0,w)$ contains a state from $F\}$

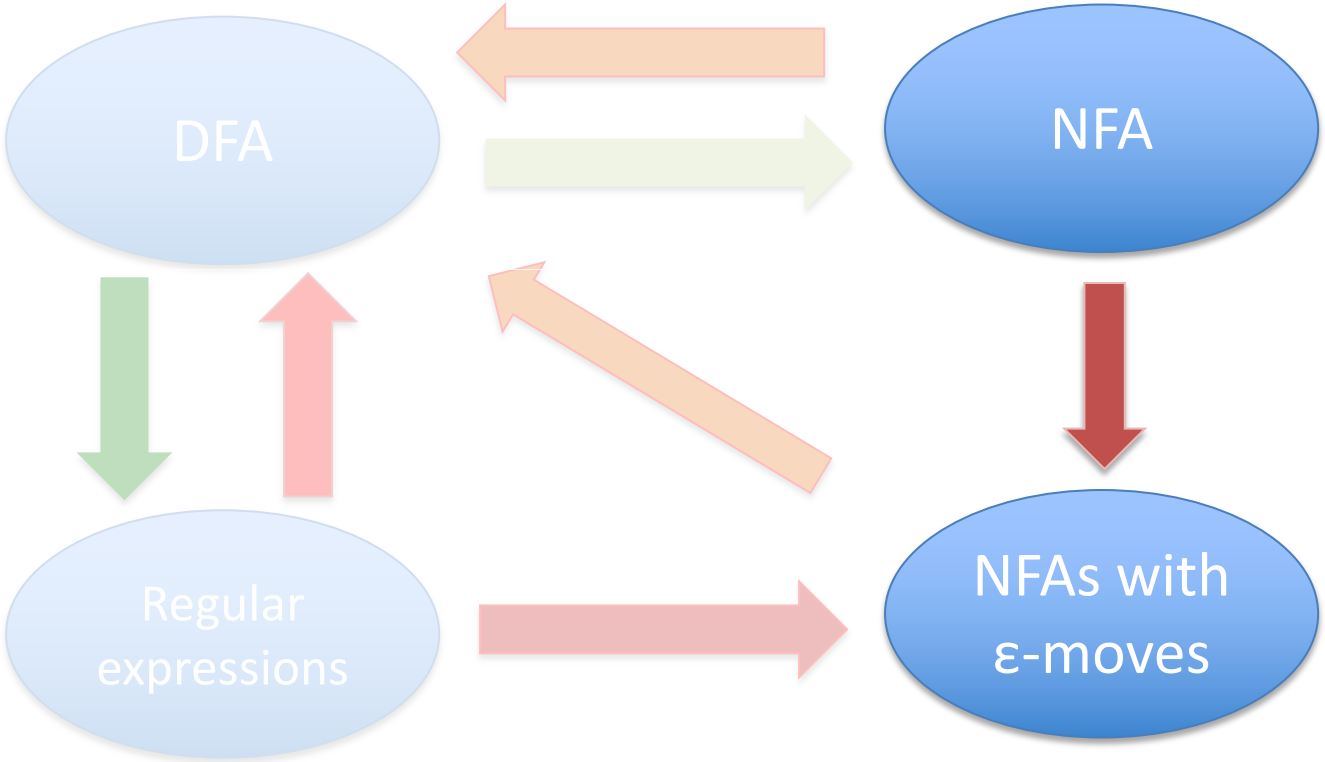
Try out

Consider the following ε-NFA:

	ε	a	b	c
→p	Ø	{p}	{q}	{r}
q	{p}	{q}	{r}	Ø
*r	{q}	{r}	Ø	{p}

compute the ε-closure of each state

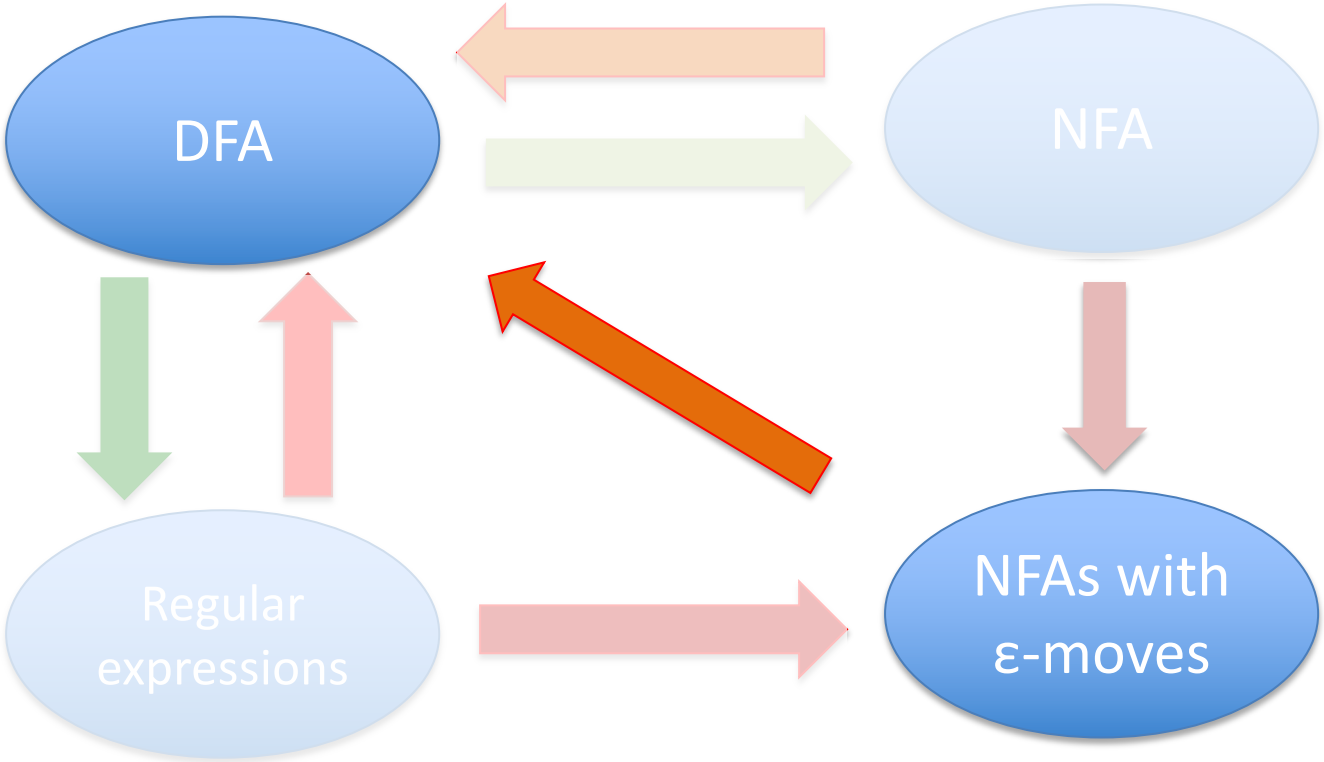
The big picture



From NFA to ε-NFA

- Any language accepted by an NFA is also accepted by an ε-NFA
  - This is trivial – but why?

The big picture



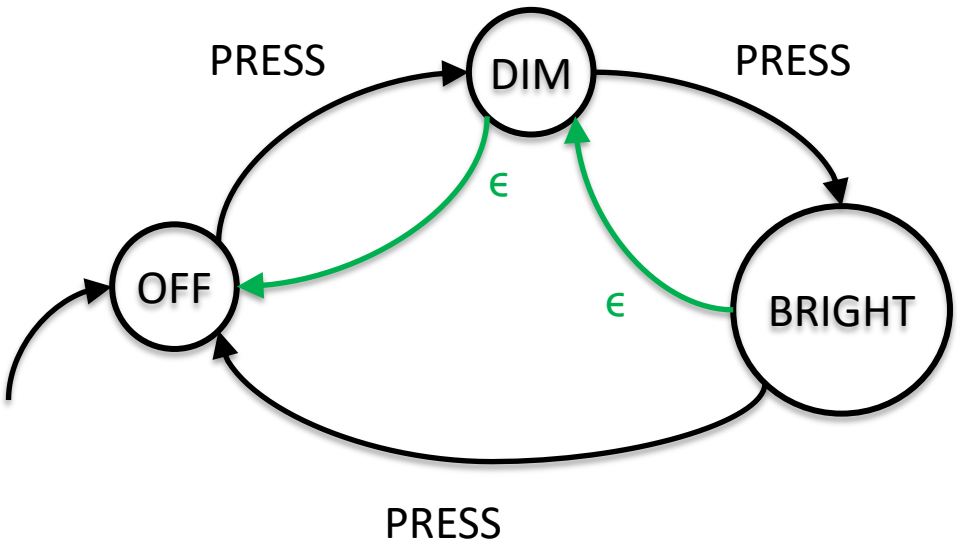
From ε-NFA to DFA

- We shall mimic the subset construction
- Given a ε-NFA $(Q_E, \Sigma, \delta_E, q_{E0}, F_E)$
- We shall construct a DFA $(Q_D, \Sigma, \delta_D, q_{D0}, F_D)$:
  - $Q_D = P(Q_E)$ – the subsets of Q_E
  - $q_{D0} = \text{ECLOSE}(q_{E0})$
  - F_D: all subsets of Q_E containing states from F_E
  - Transition function:

$$\delta_D(\{q_1,\ldots,q_k\},a) = \underline{\text{ECLOSE}}(\ \delta_E(q_1,a) \cup \ldots \cup \delta_E(q_k,a)\ )$$

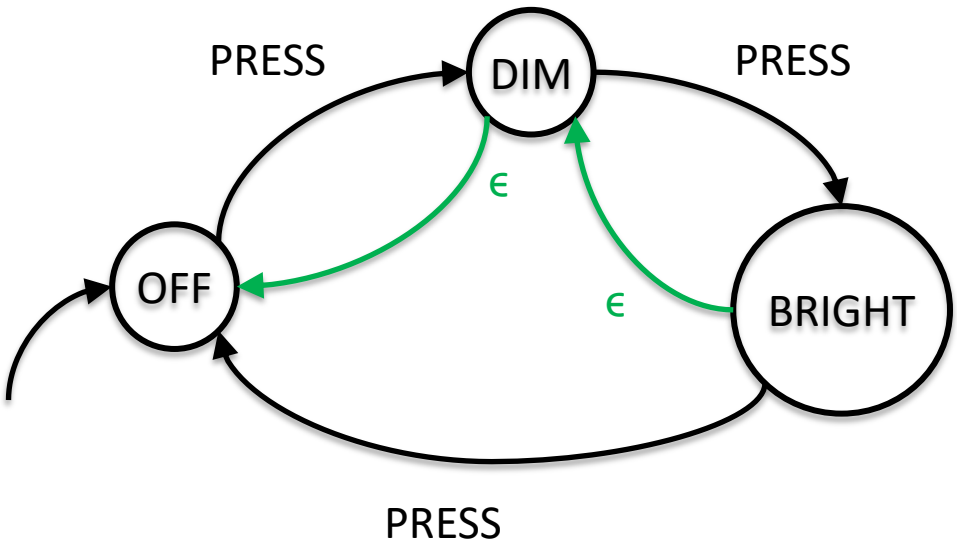
Example

The ε-NFA

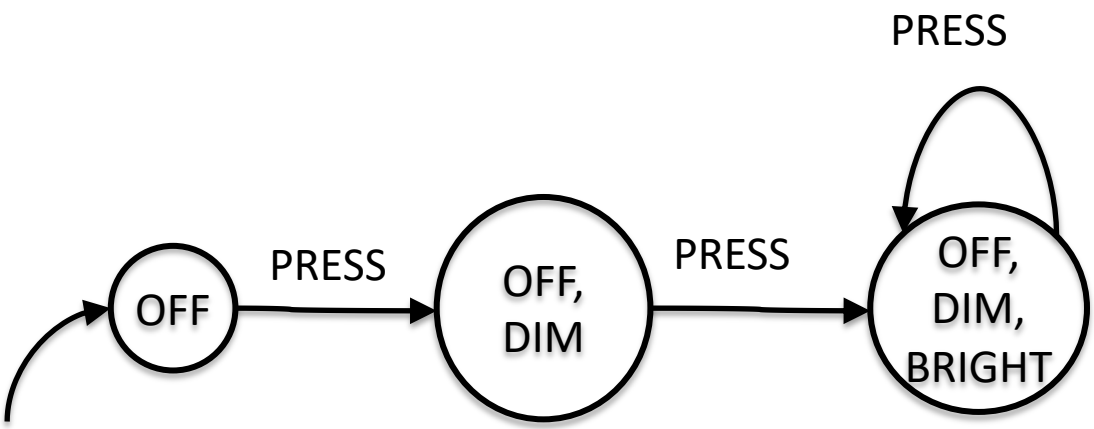


Example

The ε-NFA



Is transformed into the DFA



Try out

- Consider the following ε-NFA:

	ε	a	b	c
→p	Ø	{p}	{q}	{r}
q	{p}	{q}	{r}	Ø
*r	{q}	{r}	Ø	{p}

a) compute the ε-closure of each state

b) give all the strings of length two or less accepted by the automaton

c) convert the automaton to a DFA